Laboratory work No 2

Triggers and registers

Content

1. ***Asynchronous Triggers*** **1**
2. ***Synchronous Triggers*** **3**
   - *2.1 Static triggers* 3
   - *2.2 Two-stage triggers* 7
   - *2.3 Dynamic triggers* 9
3. ***Registries*** **10**
   - *3.1. Parallel registers* *11*
   - *3.2. The boost (sequential) register* *12*
   - *3.3. Arithmetic boosts* *12*

Logic devices and control devices use two steady-state logic elements called **triggers**. These are the simplest memory elements designed to store one bit of information (*trigger)*. In practice, SR, JK, D and T triggers are commonly used.

Asynchronous triggers respond to inputs as they occur. ***Synchronous triggers*** respond to input signals only when a synchronising (clock) signal (denoted by the symbol ***C***) appears at the corresponding input.

Asynchronous triggers

Let's meet the SR trigger. An asynchronous SR trigger has two inputs. Input ***S*** (*Set)* sets the trigger output (often called the trigger set) to the logic unit state. Input ***R*** (*Reset)* sets the trigger to the logic zero state. The trigger output ***Q*** is called the direct (or unit) output. Q_t denotes the current value of the trigger output and Q_{t+1} denotes the value of the trigger output at a subsequent point in time. Triggers often have an inverse output $\overline{Q}$.

The following is a table of the correctness (switching) of the asynchronous SR trigger.

Table 1. SR trigger switching

S	*R*	Q_t	Q_{t+1}	*Explanation*
0	0	0	0	Storage of information
0	0	1	1	
0	1	0	0	Logical zero entry
0	1	1	0	

1	0	0	1	Logical unit recording
1	0	1	1	
1	1	0	x	Prohibited combination of input signals
1	1	1	x	

From the correctness table, the SR trigger equation can be constructed. The equation describes the state of the trigger at a subsequent time instant Q_{t+1} :

$$Q_{t+1} = \bar{S}\bar{R}Q_t + S\bar{R}Q_t + S\bar{R}\bar{Q}_t = \bar{S}\bar{R}Q_t + S\bar{R} = \bar{R}(\bar{S}\,Q_t + S) = \bar{R}(Q_t + S) = \bar{R}Q_t + S.$$

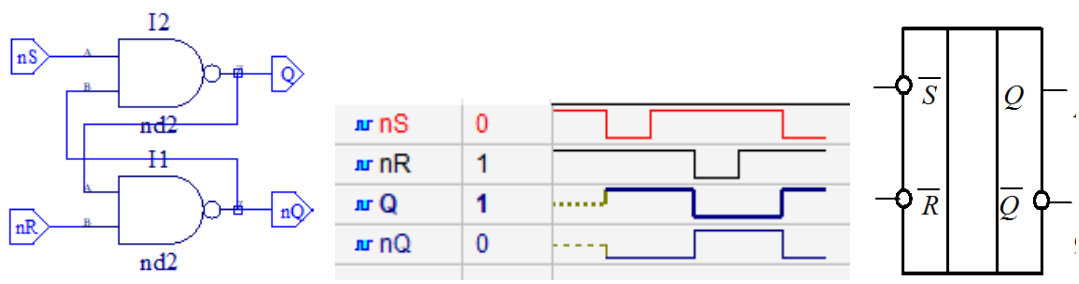
The latter expression is correct because the input signal combination $S = R = 1$ is forbidden[1] .

The equation shows that the trigger output Q will take on the value of one at a later time (Q_{t+1}) when the input S is set to logic one. The trigger output Q will take the value of a logic zero when input R is set to logic one.

The equation can be realised using only IR-NE elements. To do this, let us transform it according to De Morgan's law ($\overline{a+b} = \bar{a} \cdot \bar{b}$):

$$Q_{t+1} = \overline{\overline{Q_{t+1}}} = \overline{\overline{\bar{R}\,Q_t + S}} = \overline{\overline{\bar{R} \cdot Q_t} \cdot \bar{S}};$$

The IR-NE implementation is shown in Figure 1, a (the inputs are marked ***nS*** and ***nR*** to indicate that they are inverse).



a) b) c)

Figure 1: Schematic (a), timing diagrams (b) and graphical notation (c) of an SR trigger composed of NAND elements

Note that the trigger is in an undefined state when the storage mode is set at the start of the job (indicated by the dotted lines at the intermediate level).

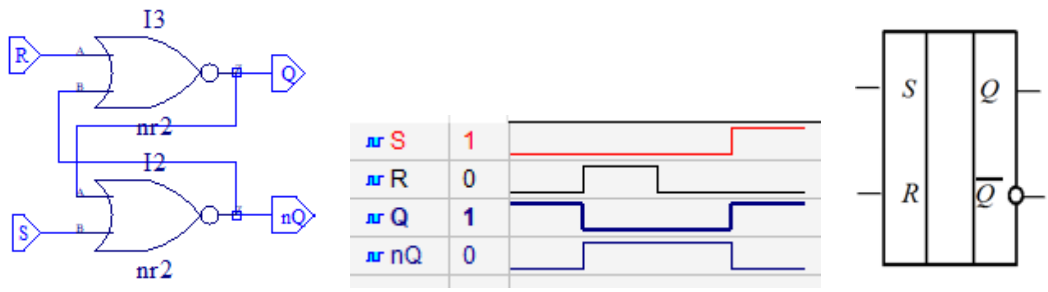
The SR trigger can also be implemented using OR-NE elements. The trigger equation is then rearranged as follows:

$$Q_{t+1} = \bar{R}\,(Q_t + S) = \overline{R + (\overline{Q + S}\,)}.$$

The implementation of the SR trigger using ARBA-NE elements is shown in Figure

[1] To see for yourself, draw a Carnot chart and minimise the expression.

2, a.



a) b) c)

Figure 2: Schematic of the SR trigger (a), consisting of ARBA-NE elements, timing diagram (b) and graphical notation (c)

Synchronous triggers

Asynchronous triggers are rarely used. Synchronous triggers are divided into the following groups:

- ***static triggers***, which have static write control and respond to input signals only when a logic unit is sent to the synchronisation input. These triggers are also known as latches;
- ***two-chip triggers*** with a two-stage memory in which a synchronisation pulse writes data to the first stage of the memory and then, when the level of the synchronisation pulse is changed, the data is overwritten from the first stage to the second stage, which produces an output signal;
- ***dynamic triggers*** controlled by a synchronisation signal front end. These triggers shall respond to the input signals only when the synchronisation pulse ***C*** moves from the logic zero level to the logic one level.

For triggers to operate reliably, the signals sent to them must meet the following requirements:

- The input signals shall remain unchanged for a period of time, known as the ***ready time*** (t_{par}), before the active synchronisation pulse front appears.
- The input signals shall maintain their values for a certain period of time, called the ***hold time*** (t_{isl}), after the active front of the synchronisation pulse.

Static triggers

We will look at the SR, UK and D triggers most commonly used in practice (T triggers are only used in meters).

Static SR trigger

The following is a table of the correctness of the SR trigger for the structure (Table 2).

Table 2. Static SR trigger switching

C	S	R	Q_t	Q_{t+1}	*Explanation*
0	x	x	0	0	Storage of information
0	x	x	1	1	
1	0	0	0	0	
1	0	0	1	1	
1	0	1	0	0	Logical zero entry
1	0	1	1	0	
1	1	0	0	1	Logical unit recording
1	1	0	1	1	
1	1	1	0	x	Prohibited combination of input signals
1	1	1	1	x	

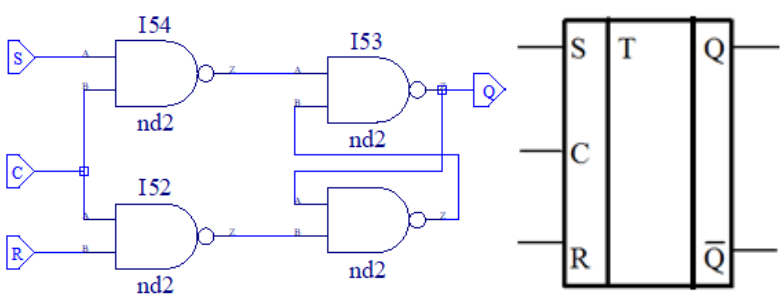
The building control trigger is described by the following equation:

$$Q_{t+1} = \bar{C}Q_t + C\bar{S}\bar{R}Q_t + CS\bar{R}\bar{Q}_t + CS\bar{R}Q_t = \bar{C}Q_t + C\bar{R}(Q_t + S) = \bar{C}Q_t + C\bar{R}Q_t + CS;$$

here $\bar{C}Q_t$ is the member describing the information storage mode (in the absence of a synchronisation pulse); $C\bar{R}(Q_t + S)$ is the member describing the operation of the trigger when a synchronisation pulse arrives at the trigger input ***C*** (***CS*** is the member obtained after minimisation of the function and evaluation of the forbidden state (***S*** = 1, ***R*** = 1)).

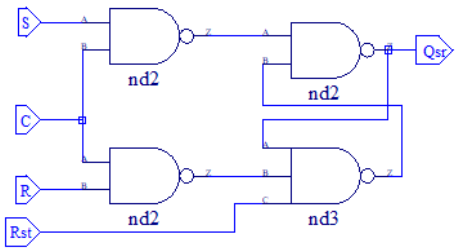
The addition of elements responding to the synchronisation pulse to the SR trigger scheme consisting of IR-NE elements results in the static SR trigger shown in Figure 3, a.

It is not possible to determine at the initial point in time which state the trigger is in, so an additional signal is often used to set the trigger to a known state. Such a circuit is shown in Figure 4, where this signal is named ***Rst*** . When analysing the operation of triggers, we will always use the following signal in our circuitry.

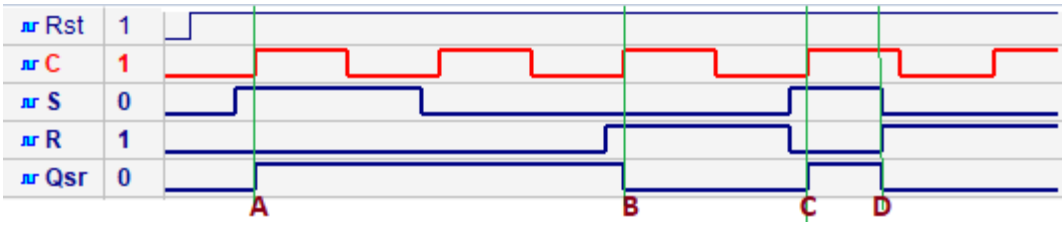


a) **b)**

Figure 3: Schematic diagram (a) and graphical representation (b) of the SR trigger for a building



a)



b)

Figure 4: Schematic (a) and timing diagrams (b) of a static SR trigger set to the initial state

The graphical representation of the static SR trigger is shown in Figure 3, b, and the timing diagram (considering the Rst signal) in Figure 4, b. In the diagram, note that the trigger only changes its state at a high level of signal ***C*** (timing D in the diagram) or when the signal goes to a high level (timing A, B, C in the diagram).

a) Static D trigger

More complex triggers consist of a simple SR memory element and a control circuit. Synchronous D triggers (*D - Data, Delay*) are widely used. They have a single information input ***D***, the information of which is remembered when the synchronisation pulse arrives and stored until the next synchronisation pulse . The inverted ***S*** signal of a synchronous SR trigger is combined with the ***R*** input to produce a D trigger as shown in Figure 5.

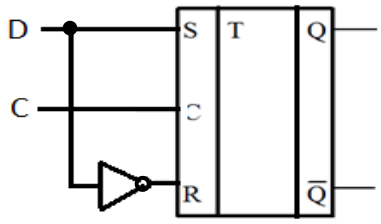


Figure 5: D trigger derived from SR trigger

The following is a table of the correctness of this trigger.

Table 3. D trigger transitions

C	*D*	Q_t	Q_{t+1}	*Explanation*

0	x	0	0	Storage of information
0	x	1	1	
1	0	0	0	Logical zero entry
1	0	1	0	
1	1	0	1	Logical unit recording
1	1	1	1	

The equation for this trigger:

$$Q_{t+1} = \overline{C}Q_t + CD_t\ ;$$

here the member $\overline{C}Q_t$ indicates that in the absence of pulse C the information is stored and the member CD_t indicates that in the presence of a sync pulse the trigger state depends on the signal level at input D.

The graphical notation and time charts are shown in Figure 6.

b) UK static trigger

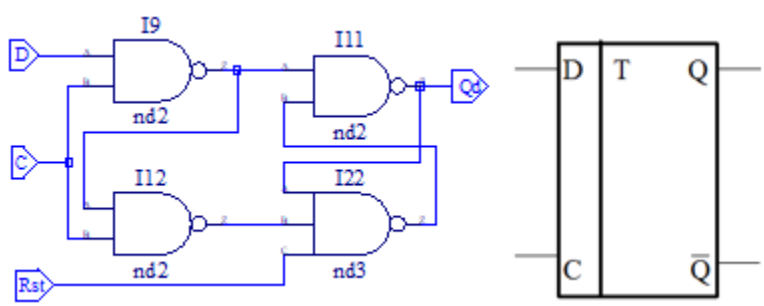
The UK trigger is versatile because it can be converted into any other type of trigger. Below is a table of the UK trigger correctness (Table 4).

The characteristic equation of a UK trigger:

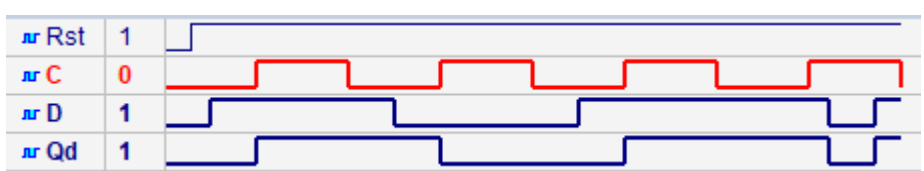
$$Q_{t+1} = \overline{K}Q_t + J\overline{Q}_t\ .$$

The characteristic equation of a synchronous JK trigger:

$$Q_{t+1} = \overline{C}Q_t + C(\overline{K}Q_t + J\overline{Q}_t\ ).$$



a) b)



c)

Figure 6: Synchronous D-trigger schematic (a), graphical notation (b), timing diagrams (c)

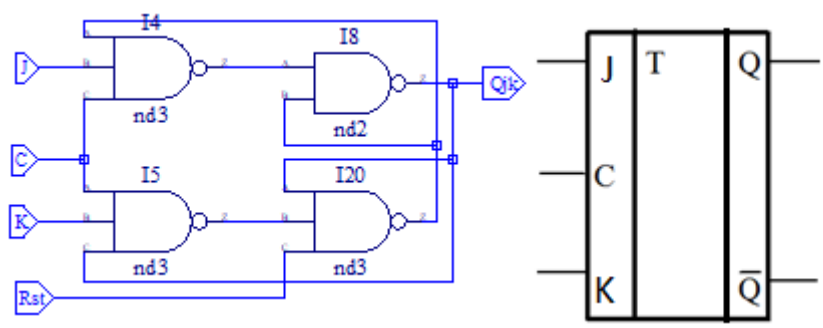
Table 4. UK trigger transitions

J	*K*	Q_t	Q_{t+1}	*Explanation*
0	0	0	0	Storage of information
0	0	1	1	
0	1	0	0	Logical zero entry
0	1	1	0	
1	0	0	1	Logical unit recording
1	0	1	1	
1	1	0	1	Setting the opposite status[2]
1	1	1	0	

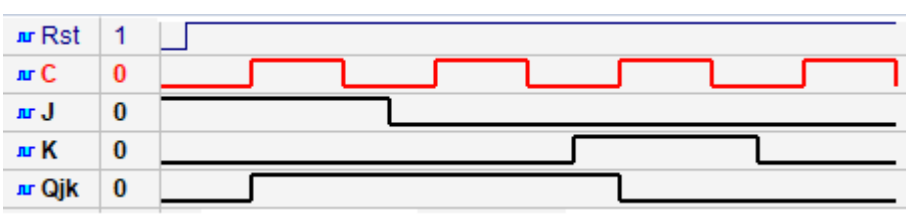
The basis of this trigger scheme is the SR trigger, augmented with feedback, as shown in Figure 7.

Two-stage triggers

A two-stroke (two-stage) trigger scheme is used to eliminate the effects of disturbances occurring at the problem input. A two-stage trigger consists of two triggers and an inverter. The triggers are connected in series; the first is synchronised by a direct synchronisation pulse and the second by an inverted synchronisation pulse as shown in Figure 8.



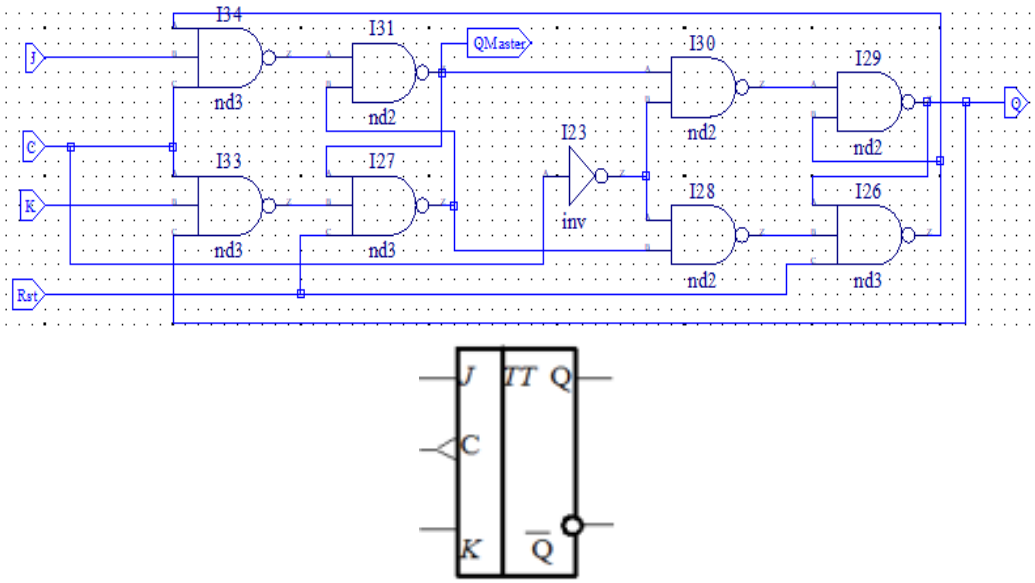
a) b)



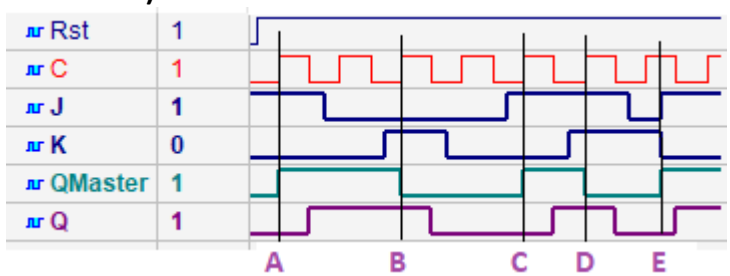
c)

[2] We note that the simulation system does not allow the static trigger to be set to ***J*** = ***K*** = **1**.

Figure 7. UK trigger diagram (a), graphical notation (b), timing diagrams (c)



a) b)



c)

Figure 8: Schematic of a two-stage UK trigger (a), graphical notation (b), timing diagrams (c)

A two-stage trigger can be implemented using any type of trigger. Since the output of the first trigger changes the state of the second trigger, the first trigger is called the *master*, **M***,* and the second trigger is called the *slave*, **S**. Such triggers are often called MS triggers. The master and slave triggers receive information at different synchronous signal levels. The information at the output is only changed after the synchronization signal has ended and is independent of the information signals at the trigger inputs at that time. The synchronization pulse determines the state of trigger M depending on the values of the information signals ***J*** and ***K, the*** state of trigger **S** does not change at that time . This structure protects the trigger against input interference.

Two T's are used to denote a two-stage MS trigger. There are SR, JK and D two-stage triggers.

The two-stage JK trigger is shown in Figure 8, a, its graphical notation in Figure 8, b, and its timing diagrams in Figure 8, c. In the diagram, note that the first trigger (trigger

M) only changes its state at a high level of signal ***C*** (timing E in the diagram), or as the signal goes to a high level (timing A, B, C, D in the diagram). The second trigger (trigger S) changes its state when signal ***C*** goes low.

Dynamic triggers

Dynamic triggers are often used in practice. These triggers react to a change in the level of the synchronisation signal (signal front).

There are SR, JK and D types of dynamic triggers. A schematic of an SR dynamic trigger is shown in Figure 9.

Elements I6 and I17 form the main (output) trigger, while I4 and I37, I36 and I5 form the auxiliary (commutation) triggers. The information from the auxiliary triggers is overwritten by the main trigger when the synchronisation pulse goes from logic one to zero.

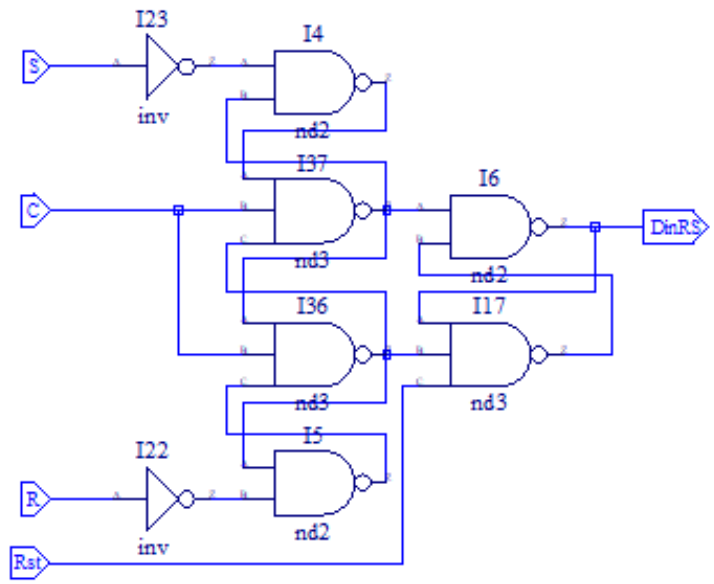


Figure 9: Schematic of SR dynamic trigger

A closer look at the operation of this trigger reveals a fundamental flaw: when low-level ***S*** and ***R*** signals arrive at the leading edge of the synchronization pulse, and then change the values of the ***S*** and ***R*** signals before the end of the synchronization pulse, the trigger responds to these changes with a change of state. For this reason, type D dynamic triggers are commonly used. A schematic of such a trigger is shown in Figure 10.

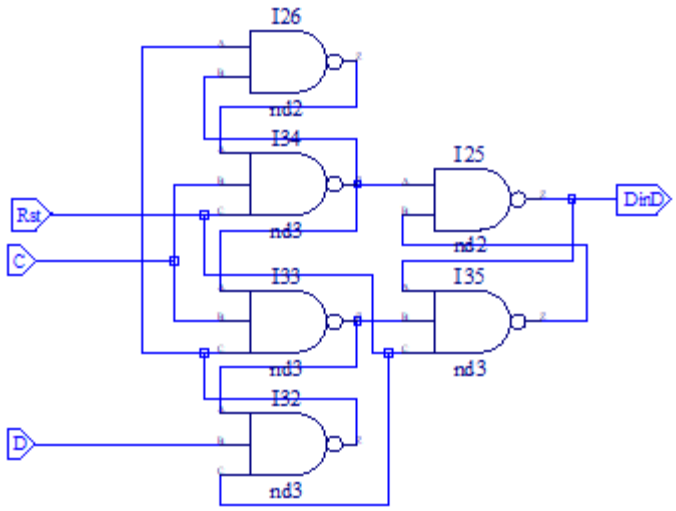


Figure 10: Schematic of dynamic trigger D with static zeroing input (*Rst*)

UK dynamic triggers are also available (e.g. 74ALS109). Such a trigger does not have the disadvantage of the SR trigger mentioned above, due to the feedback from leads ***Q*** and ***nQ*** in the JK trigger. A schematic of a JK dynamic trigger is given in Figure 11.

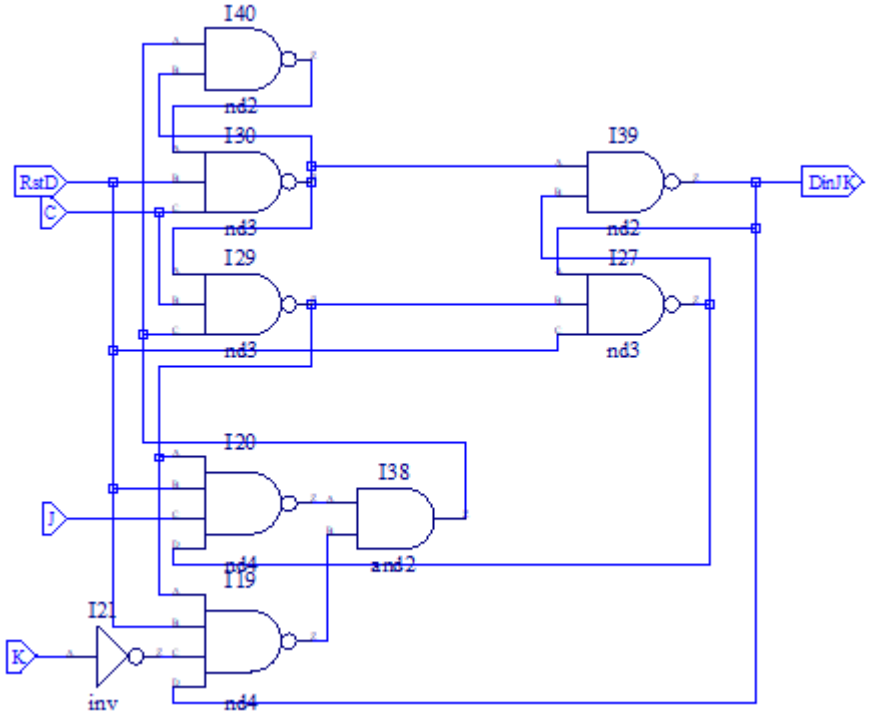


Figure 11. Schematic of a UK dynamic trigger

Registries

A register is a device for recording, storing and reading information, as well as for

performing certain other operations on information. A register may store more than one bit of information.

Registers consist of memory elements (triggers) and the circuitry that controls them. The length of a register, or the number of columns, is the number of triggers that make up the register.

The registry can be:

- storage (parallel);
- boost (sequential);
- universal.

The register can be single-beat (where dynamic triggers are commonly used) or double-beat, where information is written over two beats (as in MS triggers). Since the lowest column (zero) of a word or byte of information is usually on the right, we will start numbering the columns of information stored in registers on the right-hand side (sometimes reversed in the literature). In this case, the numeric code stored in the register can be translated into decimal by raising the binary by the power of the index (column number).

3.1. Parallel registers

The parallel register consists of *n* synchronous D-triggers with input and output control circuits, as shown in Figure 12.

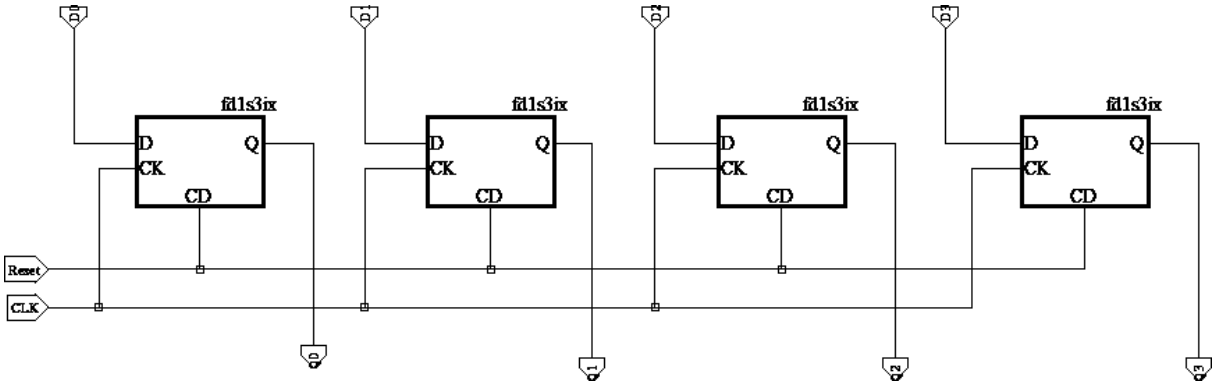


Figure 12: Four-column parallel register with control circuits

The outputs of the D triggers used in this register are marked with the symbol **Q** and the inputs with the symbols:

- **D** is the input for the information to be recorded;
- **CK** is the synchronisation signal; if dynamic triggers are used in the register, the information will be recorded at the leading edge of the signal ***CLK***;

- **CD** - reset signal.

3.2. The boost (sequential) registers i

The push registers can be right (*LR*), left (*LL*) or right and left (universal). During a push, the input information is sequentially pushed along the register by a synchronisation signal until it reaches the register output.

By connecting the synchronous dynamic triggers in series, we get a boost register that performs a logic boost to the right. A schematic of such a register is shown in Figure 13.

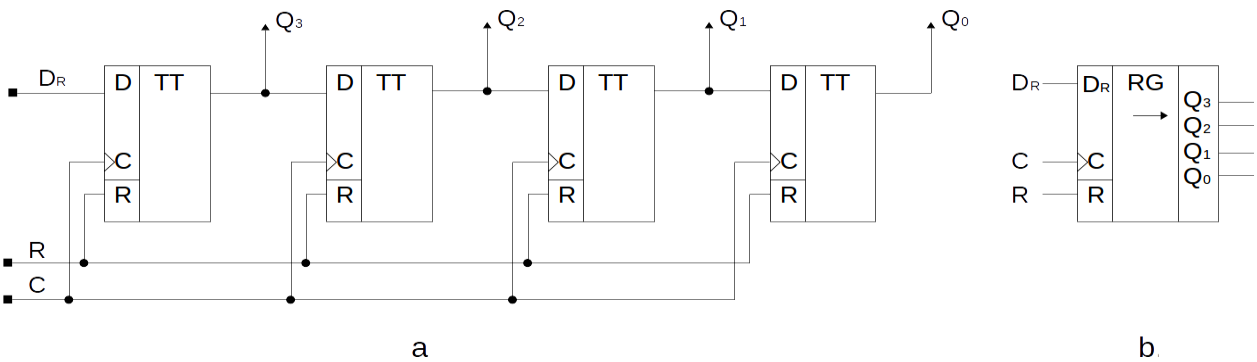


Figure 13: Schematic (a) and graphical representation (b) of the four-column right-bias register

There are three types of boost:

1. **A logical boost** (*LL*, *LR*) where the value fed to a special input (D_L , D_R) is written into the columns that are to be filled;
2. **Cyclical boost** (*CR*, *CL*), where information from the columns that have been displaced is entered into the columns that are emptying;
3. **Arithmetic advance** (*AR*, *AL*), performed without pushing the highest (sign) column, thus preserving the sign of the number. A one-column shift to the right (AR_1) of the remaining columns means a reduction of the absolute value of the number by a factor of two, and to the left an increase by a factor of two. The arithmetic advance is performed by entering 0 or 1 in the free columns, depending on the number code and sign.

3.3. Arithmetic boosts

The sign of the number is usually indicated by the value in the top column (0 for a positive number, 1 for a negative number). On computers, positive numbers are coded in the same way, while the coding of negative numbers depends on the code chosen.

There are three known methods (codes) for representing negative numbers:

1. *Sign-and-magnitude* code;
2. Reverse code (*Ones' complement)*;
3. *Two's complement* code.

The most common and almost all general-purpose processors currently in use use use add-on code.

One bit, usually the high bit, is used to store the number sign. If this bit is 0, the number is positive and 1 is negative. The remaining bits represent the absolute value (magnitude) of the number. Thus, if a word is 8 bits long, one bit is allocated for the sign and 7 bits are left to store the value. The value of a positive number can vary from 0000000 to 1111111 (127).

A reminder of the rules for encoding negative numbers.

Direct code. In direct code, the code for a negative number (-A) differs from the code for a number (+A) only by the sign column. For example, in direct code:

+26 :00011010, -26 :10011010.

Interestingly, we have two codes to represent the number 0: 00000000 (+0) and 10000000 (-0).

Reverse code. It represents negative numbers as the inverse of bits of positive numbers. For example, in reverse code:

+26 :00011010, -26 :11100101

As in the direct code, in the reverse code we also have two codes to represent the number 0: 00000000 (+0) and 11111111 (-0).

Additional code. In the supplementary code, negative numbers are represented by a code that is one higher than the inverse of the numeric code. There is only one code to represent the number 0 in the supplementary code (00000000). The advantage of the additive code is that any number (positive or negative) adds up equally and the sum is also obtained in the additive code.

For example, +1 in binary is 000000001. Invert and add 1 to get the code for the number -1: $\overline{00000001}$ + 1 = 11111110 + 1 = 11111111 (-1 in the additional code).

This system is useful for simplifying calculations in hardware: it allows negative numbers to be added without subtraction and without any additional correction steps. There is also no need for logic to determine the sign of the sum.

When information is pushed to the registers, the information shown in Table 5 shall be entered in the columns that become available.

Table 5. Filling in the blanks for arithmetic boosts

Source :	***Code***	***Enter in the box that opens***	
		by pushing to the right	***by pushing to the left***
Positive	any	0	0
negative	direct	0	0
	reverse	1	1
	additional	1	0